<!-- به نام خدا -->

مهندسی اینترنت، فصل ۱: پروتکل‌ها

واسطه‌ها: cache و proxy و CDN

جلسهٔ ۵: چه کسی بین شما و سرور نشسته است، و چه چیزی را می‌بیند؟

parse · HSTS · DNS · اتصال · TLS ◌ · HTTP ●

محمد بصیرزاده

دانشگاه شهید چمران اهواز، دانشکدهٔ مهندسی، پاییز ۱۴۰۵

> در جلسهٔ ۴ دیدیم یک درخواست تازهٔ HTTPS چند رفت‌وبرگشت تا سرور طول می‌کشد؛ امروز مشکل، تعداد درخواست‌هاست.

روز انتخاب واحد است؛ سامانهٔ آموزش را چگونه سرپا نگه داریم؟

- ساعت ۸ صبح، ۲۰ هزار دانشجو در یک دقیقه صفحهٔ اصلی سامانه را باز می‌کنند: بیش از ۳۰۰ درخواست در ثانیه
- صفحه برای همه یکسان است، ولی سرور در هر ثانیه فقط ۲۰۰ صفحه می‌سازد. بودجه‌ای برای سرور تازه نیست؛ چه می‌کنید؟

در جلسهٔ ۴ دیدیم یک درخواست تازهٔ HTTPS چند رفت‌وبرگشت تا سرور طول می‌کشد؛ امروز مشکل، تعداد درخواست‌هاست.

روز انتخاب واحد است؛ سامانهٔ آموزش را چگونه سرپا نگه داریم؟

- ساعت ۸ صبح، ۲۰ هزار دانشجو در یک دقیقه صفحهٔ اصلی سامانه را باز می‌کنند: بیش از ۳۰۰ درخواست در ثانیه
- صفحه برای همه یکسان است، ولی سرور در هر ثانیه فقط ۲۰۰ صفحه می‌سازد. بودجه‌ای برای سرور تازه نیست؛ چه می‌کنید؟

یک پاسخ: صفحه‌ای که برای همه یکسان است، لازم نیست ۲۰ هزار بار ساخته شود. یک بار بسازید و نسخه‌ای از آن را نزدیک دانشجوها نگه دارید. به این نسخه **cache** می‌گویند و به نرم‌افزاری که بین مرورگر و سرور آن را نگه می‌دارد، **واسطه**.

> **سریع‌ترین درخواست آن است که اصلاً به سرور نرسد.**

پاسخی که دریافت کردید، از سرور اصلی آمده یا از یک cache؟

سؤال اگر یک صفحه را دو بار پشت‌سرهم درخواست کنیم، پاسخ دوم را چه کسی می‌فرستد؟

مشاهده در هر دو اجرا به فیلد `age` و به `x-cache` (یا `cf-cache-status`) نگاه کنید: پاسخ از cache آمده است (hit) یا نه (miss)؟

تصمیم پاسخی که برای همهٔ کاربران یکسان است، می‌تواند در سروری نزدیک کاربر بماند تا درخواست اصلاً به سرور اصلی نرسد.

Windows | Linux / macOS

```
U=https://www.wikipedia.org/
for i in 1 2; do
  curl -sI $U | grep -iE '^(age|cache-control|x-cache|cf-cache-status)'
  echo; sleep 2
done
```

گزینهٔ `-I` فقط headerهای پاسخ را دریافت می‌کند. همین دستور را روی سایت دیگری هم اجرا کنید؛ هر CDN برای اعلام hit نام فیلد متفاوتی دارد.

خروجی دمو را چگونه بخوانیم؟

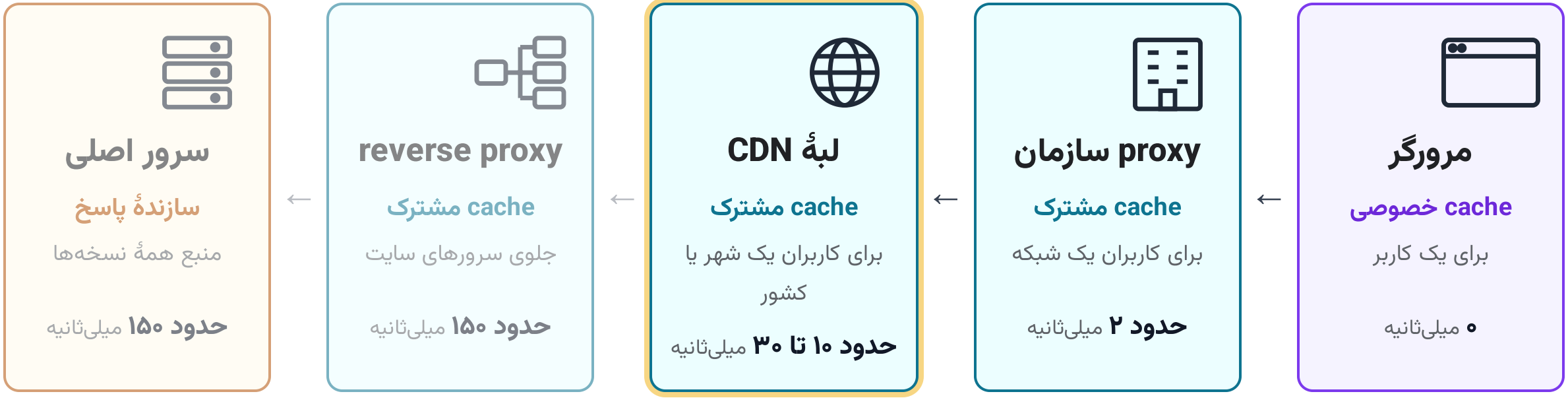
- فیلد Age استاندارد است و می‌گوید از زمانی که سرور اصلی پاسخ را ساخته، چند ثانیه گذشته است. اگر این فیلد را می‌بینید، **پاسخ از یک cache آمده است**
- فیلدهای `X-Cache` و `CF-Cache-Status` استاندارد نیستند؛ هر CDN نام و قالب خودش را دارد
- اگر شبکهٔ شما proxy دارد، ممکن است همان proxy فیلد `Age` را اضافه کرده باشد

```
cache-control: public, max-age=30
etag: "eeabd5f1e598"
x-cache: MISS

cache-control: public, max-age=30
etag: "eeabd5f1e598"
x-cache: HIT
age: 2
```

این خروجی از اجرای همان دستور روی سرور و cache محلی پوشهٔ `labs/05-intermediaries/local` به دست آمده است.

درخواست شما از چند لایهٔ cache می‌گذرد؟



درخواست تا جایی جلو می‌رود که یکی از لایه‌ها نسخهٔ معتبری داشته باشد؛ در این شکل لبهٔ CDN پاسخ را دارد و بقیهٔ راه حذف می‌شود. در این جلسه برای هر لایه چهار پرسش را پاسخ می‌دهیم: پاسخ را نگه می‌دارد؟ برای همه یا یک نفر؟ به نمایندگی از چه کسی؟ متن ساده را می‌بیند؟

> **هر لایه‌ای که نسخهٔ معتبر داشته باشد، بقیهٔ راه تا سرور اصلی حذف می‌شود.**

بخش ۱ از ۵

cache و تاریخ مصرف پاسخ

تا اینجا: دیدیم پاسخ ممکن است از یکی از لایه‌های cache برگردد، نه از سرور اصلی.

سؤال این بخش

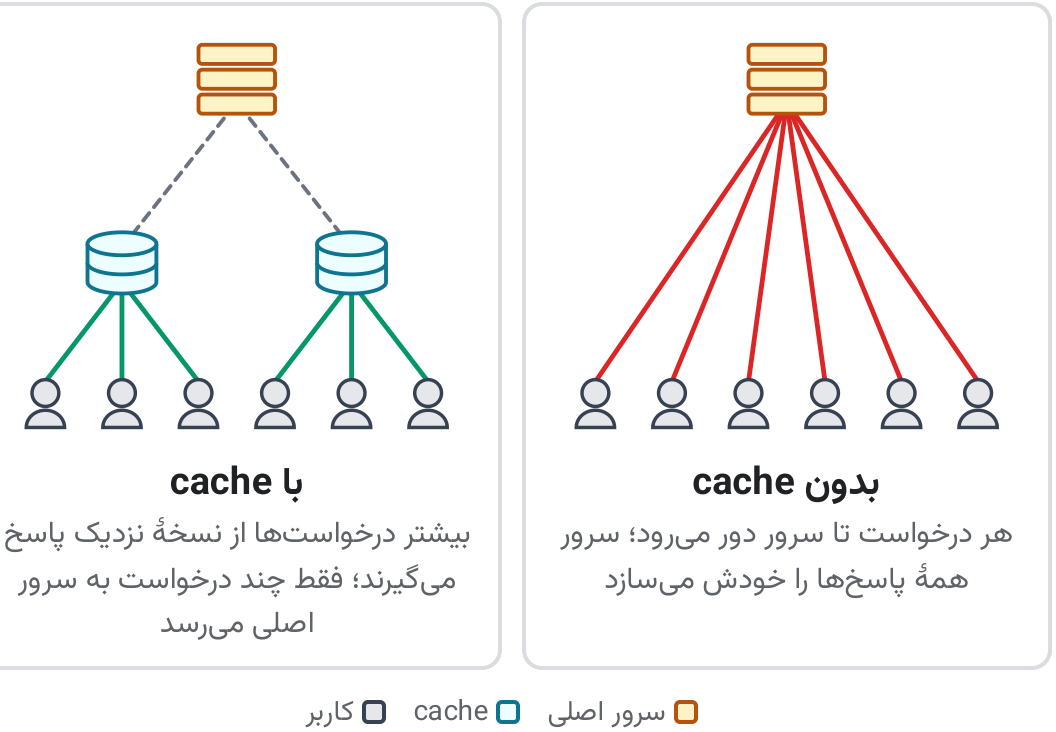
cache تا چه زمانی می‌تواند پاسخی را بدون پرسیدن از سرور تحویل دهد، و این را چه کسی تعیین می‌کند؟

05-intermediaries.todo

- [ ] cache و تازگی
- [ ] conditional request
- [ ] کلید cache
- [ ] proxy و CDN
- [ ] جمع‌بندی

cache چه مشکلی را حل می‌کند و چه مشکلی می‌سازد؟

- **حل می‌کند:** پاسخ از نزدیک می‌آید، سرور یک پاسخ را برای هزاران درخواست یک بار می‌سازد، و پهنای باند کمتری مصرف می‌شود
- **می‌سازد:** نسخهٔ ذخیره‌شده ممکن است **کهنه باشد** یا به کاربر اشتباهی برسد، و پاک کردنش از همهٔ لایه‌ها دشوار است
- سرور در header هر پاسخ به cacheها می‌گوید آن را تا کی و برای چه کسی نگه دارند



cache سرعت می‌دهد، ولی خطر پاسخ کهنه یا نادرست را هم می‌آورد.

پهنای باند بیشتر یا cache: کدام صفحه‌ها را زودتر باز می‌کند؟

- سرورها دورند (هر رفت‌وبرگشت حدود ۲ ثانیه) و اینترنت دانشکده در ساعت‌های شلوغ پر است
- پیشنهاد اول: پهنای باند صد برابر، با هزینهٔ زیاد. پیشنهاد دوم: cacheی که ۴۰٪ درخواست‌ها را پاسخ دهد

الف) پیشنهاد اول

ب) پیشنهاد دوم

ج) هر دو یکسان

د) هیچ‌کدام

پهنای باند بیشتر یا cache: کدام صفحه‌ها را زودتر باز می‌کند؟

- سرورها دورند (هر رفت‌وبرگشت حدود ۲ ثانیه) و اینترنت دانشکده در ساعت‌های شلوغ پر است
- پیشنهاد اول: پهنای باند صد برابر، با هزینهٔ زیاد. پیشنهاد دوم: cacheی که ۴۰٪ درخواست‌ها را پاسخ دهد

الف) پیشنهاد اول	ب) پیشنهاد دوم	ج) هر دو یکسان	د) هیچ‌کدام

پاسخ: ب) با پهنای باند بیشتر هم سرورها ۲ ثانیه دورند؛ اما cache حدود ۴۰٪ درخواست‌ها را در دانشکده پاسخ می‌دهد و میانگین را به ۱٫۲ ثانیه می‌رساند.

> **همان درس جلسهٔ ۴: زمان را فاصله تعیین می‌کند، نه پهنای باند.**

سرور چگونه می‌گوید پاسخ تا کی تازه است؟

- سرور در پاسخ می‌نویسد max-age=600: این پاسخ تا ۶۰۰ ثانیه پس از ساخته شدن، تازه (fresh) است
- cache مشترک در فیلد `Age` می‌گوید از ساخته شدن پاسخ چند ثانیه گذشته است. تا وقتی `Age` کمتر از `max-age` است، پاسخ **بدون پرسیدن از سرور** تحویل داده می‌شود؛ در این مثال تا ۴۸۰ ثانیهٔ دیگر
- پس از آن پاسخ کهنه (stale) است؛ دور ریخته نمی‌شود، ولی پیش از استفاده باید اعتبارش بررسی شود (بخش بعد)

```
HTTP/1.1 200 OK
Cache-Control: max-age=600
Age: 120
Content-Type: text/html
```

سرور تاریخ مصرف پاسخ را تعیین می‌کند و cache آن را رعایت می‌کند.

هر لایهٔ cache از کدام دستور پیروی می‌کند؟

دستور در پاسخ سرور	cache مرورگر (خصوصی)	cache مشترک (proxy و CDN)
`max-age=600`	۱۰ دقیقه نگه می‌دارد	۱۰ دقیقه نگه می‌دارد
`private`	نگه می‌دارد	**نگه نمی‌دارد**
`s-maxage=3600`	این دستور را نادیده می‌گیرد	یک ساعت نگه می‌دارد
`no-cache`	نگه می‌دارد، ولی پیش از هر استفاده می‌پرسد	نگه می‌دارد، ولی پیش از هر استفاده می‌پرسد
`no-store`	نگه نمی‌دارد	نگه نمی‌دارد

چند دستور را می‌توان با هم نوشت، مثل `Cache-Control: max-age=60, s-maxage=3600` : مرورگر یک دقیقه و CDN یک ساعت نگه می‌دارد.

> **با یک header، سرور برای هر لایه جداگانه تصمیم می‌گیرد.**

کدام دستورها برای سه پاسخ سامانهٔ بوفه درست‌اند؟

گزینه	منوی روز (برای همه یکسان)	موجودی کیف پول (مخصوص هر دانشجو)	لوگو (ماه‌ها ثابت)
الف)	`no-cache`	`no-cache`	`no-cache`
ب)	`public, max-age=300`	`private, no-cache`	`max-age=86400`
ج)	`no-store`	`max-age=60`	`no-store`
د)	`max-age=300`	`max-age=300`	`max-age=300`

کدام دستورها برای سه پاسخ سامانهٔ بوفه درست‌اند؟

گزینه	منوی روز (برای همه یکسان)	موجودی کیف پول (مخصوص هر دانشجو)	لوگو (ماه‌ها ثابت)
الف)	`no-cache`	`no-cache`	`no-cache`
ب)	`public, max-age=300`	`private, no-cache`	`max-age=86400`
ج)	`no-store`	`max-age=60`	`no-store`
د)	`max-age=300`	`max-age=300`	`max-age=300`

پاسخ: ب) موجودی با `private` فقط در مرورگر خود دانشجو می‌ماند و با `no-cache` پیش از هر نمایش، اعتبارش از سرور پرسیده می‌شود. در ج و د موجودی یک نفر به دیگران می‌رسد.

> **دستور no-cache یعنی «پیش از استفاده بپرس»؛ برای «ذخیره نکن» باید no-store نوشت.**

اگر سرور تاریخ مصرف ننویسد، cache چه می‌کند؟

- بدون `max-age` و `Expires`، cache خودش عمری حدس می‌زند، معمولاً **۱۰٪ زمانِ گذشته از آخرین تغییر**: فایلی که ده روز پیش تغییر کرده، حدود یک روز تازه است (RFC 9111 §4.2.2)
- فیلد `Expires` روش HTTP/1.0 است و به‌جای «چند ثانیه» یک تاریخ می‌دهد، مثل `Expires: Wed, 05 Nov 2026 08:00:00 GMT`؛ اگر `max-age` هم باشد، `max-age` ملاک است
- با `public`، cache مشترک حتی پاسخ کاربرِ واردشده (درخواست دارای `Authorization`) را نگه می‌دارد

> **اگر تاریخ مصرف را ننویسید، cache حدس می‌زند و حدسش لزوماً درست نیست.**

پیوند با هوش مصنوعی: چرا crawlerها برای ویکی‌پدیا پرهزینه‌اند؟

- طبق گزارش ویکی‌مدیا (۱۲ فروردین ۱۴۰۴)، crawlerها حدود ۳۵٪ بازدیدها را می‌سازند، ولی ۶۵٪ پرهزینه‌ترین ترافیک از آن‌هاست
- صفحه‌های پربازدید در cache نزدیک کاربرند و ارزان تحویل داده می‌شوند. crawlerها صفحه‌های کم‌بازدید را هم می‌خوانند و این درخواست‌ها **به مرکز داده اصلی** می‌رسند
- پهنای باند دانلود چندرسانه‌ای از دی ۱۴۰۲ حدود ۵۰٪ بیشتر شده است، بیشتر برای آموزش مدل‌ها (Wikimedia Diff)

> **cache فقط به درخواست تکراری کمک می‌کند؛ crawlerی که هر صفحه را یک بار می‌خواند، به سرور اصلی می‌رسد.**

بخش ۲ از ۵

تاریخ مصرف گذشته؛ دوباره دانلود کنیم یا فقط بپرسیم؟

تا اینجا: گذشتن تاریخ مصرف یک پاسخ، به این معنا نیست که محتوای آن روی سرور تغییر کرده است.

سؤال این بخش

مرورگر نسخهٔ منقضی‌شدهٔ یک فایل ۲ مگابایتی را دارد و فایل روی سرور تغییری نکرده است. آیا باید هر ۲ مگابایت دوباره فرستاده شود؟

05-intermediaries.todo

- [x] cache و تازگی
- [>] conditional request
- [ ] کلید cache
- [ ] proxy و CDN
- [ ] جمع‌بندی

اگر پروتکل را شما طراحی کنید، cache چگونه بپرسد نسخه‌اش معتبر است؟

الف) کل نسخهٔ خودش را برای سرور بفرستد تا سرور مقایسه کند

ب) شناسهٔ کوتاهی را که سرور همراه همان نسخه داده بود، به سرور برگرداند

ج) اندازهٔ فایل را بپرسد و اگر برابر بود، نسخه را معتبر بداند

د) هر بار کل فایل را دوباره دریافت کند

اگر پروتکل را شما طراحی کنید، cache چگونه بپرسد نسخه‌اش معتبر است؟

الف) کل نسخهٔ خودش را برای سرور بفرستد تا سرور مقایسه کند

ب) شناسهٔ کوتاهی را که سرور همراه همان نسخه داده بود، به سرور برگرداند

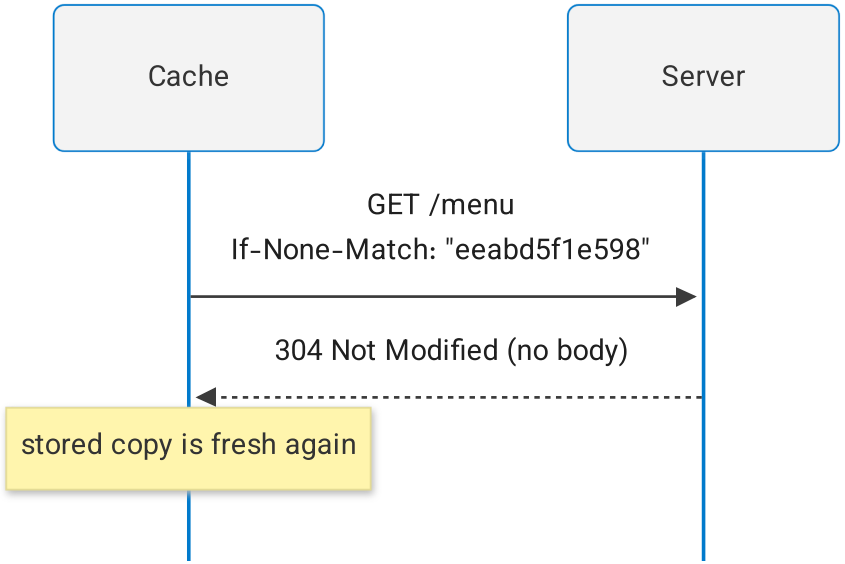
ج) اندازهٔ فایل را بپرسد و اگر برابر بود، نسخه را معتبر بداند

د) هر بار کل فایل را دوباره دریافت کند

پاسخ: ب) cache شناسهٔ نسخه را برمی‌گرداند و سرور فقط می‌گوید «همان است» یا نسخهٔ تازه را می‌فرستد. اندازهٔ برابر ثابت نمی‌کند محتوا یکسان است.

> **برای پرسیدن «هنوز همان است؟» یک شناسهٔ کوتاه کافی است.**

پرسش «هنوز همان است؟» در HTTP چگونه نوشته می‌شود؟



- سرور همراه پاسخ یک اعتبارسنج (validator) می‌فرستد: ETag (شناسهٔ نسخه) یا `Last-Modified` (زمان آخرین تغییر)
- cache همان مقدار را در `If-None-Match` یا `If-Modified-Since` برمی‌گرداند؛ کتاب این را conditional GET می‌نامد (بخش ۲.۲.۵)
- اگر منبع تغییر نکرده باشد، پاسخ `304 Not Modified` و **بدون بدنه** است؛ وگرنه `200` با نسخهٔ تازه

پاسخ 304 یعنی «نسخهٔ تو هنوز معتبر است»؛ یک RTT می‌گیرد، ولی بدنه را دوباره نمی‌فرستد.

فایل `app.js` هر هفته عوض می‌شود؛ چگونه cache شود؟

دستور immutable یعنی «این محتوا هرگز عوض نمی‌شود؛ حتی با بارگذاری دوباره نپرس».

الف) نشانی ثابت `/app.js` با `max-age=60`

ب) نشانی ثابت با `no-cache` و `ETag`

ج) نشانی تازه برای هر نسخه، با `max-age=31536000, immutable`

د) نشانی ثابت با `no-store`

فایل `app.js` هر هفته عوض می‌شود؛ چگونه cache شود؟

دستور immutable یعنی «این محتوا هرگز عوض نمی‌شود؛ حتی با بارگذاری دوباره نپرس».

الف) نشانی ثابت `/app.js` با `max-age=60`

ب) نشانی ثابت با `no-cache` و `ETag`

ج) نشانی تازه برای هر نسخه، با `max-age=31536000, immutable`

د) نشانی ثابت با `no-store`

پاسخ: ج) نشانی دارای هش، مثل `/app.3f9a2c.js`، هرگز محتوای دیگری نمی‌گیرد؛ پس یک سال بدون پرسیدن در cache می‌ماند.

> **برای فایلی که تغییر می‌کند، نشانی تازه بهتر از بررسی دوبارهٔ اعتبار است.**

اعتبارسنج ضعیف و دستورهای مربوط به پاسخ کهنه چه می‌گویند؟

- مقدار ETag با پیشوند `W/` ضعیف است: دو نسخه از نظر معنا یکسان‌اند، نه لزوماً بایت به بایت (RFC 9110 §8.8.1)
- با `stale-while-revalidate=60`، cache تا ۶۰ ثانیه پاسخ کهنه را فوراً می‌دهد و در پس‌زمینه اعتبارش را بررسی می‌کند (RFC 5861)
- با `must-revalidate`، پاسخ کهنه **حتی وقتی سرور در دسترس نیست** تحویل داده نمی‌شود
- کلاینت هم می‌تواند در درخواست `Cache-Control` بفرستد: بارگذاری دوباره درخواستی conditional با `max-age=0` می‌فرستد؛ بارگذاری اجباری، بدون اعتبارسنج و با `no-cache` (MDN)

> **این دستورها تعیین می‌کنند پاسخ تا چه حد کهنه باشد و باز هم تحویل داده شود.**

بخش ۳ از ۵

یک نشانی، چند کاربر: این پاسخ مال کیست؟

تا اینجا: تا اینجا پرسیدیم پاسخ «تا کی» معتبر است؛ حالا می‌پرسیم «برای چه کسی».

سؤال این بخش

سارا و رضا هر دو نشانی /balance را باز می‌کنند. اگر cache پاسخ سارا را نگه داشته باشد، از کجا بفهمد نباید آن را به رضا بدهد؟

05-intermediaries.todo

- [x] cache و تازگی
- [x] conditional request
- [>] کلید cache ▶
- [ ] proxy و CDN
- [ ] جمع‌بندی

سناریوی واقعی: چرا کاربران Steam حساب دیگران را دیدند؟

در کریسمس ۲۰۱۵، Steam زیر حملهٔ DDoS بود و پیکربندی تازه‌ای روی cache لبه اعمال شد. سپس صفحهٔ حساب ۳۴ هزار کاربر به دیگران نشان داده شد. چرا؟

الف) مهاجمان به حساب‌ها نفوذ کرده بودند

ب) cache صفحه‌های شخصی را به دیگران داد

ج) مرورگرها کوکی اشتباه فرستادند

د) پایگاه‌داده رکوردها را جابه‌جا برگرداند

سناریوی واقعی: چرا کاربران Steam حساب دیگران را دیدند؟

در کریسمس ۲۰۱۵، Steam زیر حملهٔ DDoS بود و پیکربندی تازه‌ای روی cache لبه اعمال شد. سپس صفحهٔ حساب ۳۴ هزار کاربر به دیگران نشان داده شد. چرا؟

الف) مهاجمان به حساب‌ها نفوذ کرده بودند	**ب)** cache صفحه‌های شخصی را به دیگران داد	**ج)** مرورگرها کوکی اشتباه فرستادند	**د)** پایگاه‌داده رکوردها را جابه‌جا برگرداند

پاسخ: ب) به گفتهٔ Valve، صفحه‌های کاربرانِ واردشده هم با کلیدی ذخیره شدند که فقط نشانی صفحه بود.

> **پاسخ شخصی که خودش نگوید private، در cache مشترک به دیگران هم می‌رسد.**

کلید cache از چه چیزی ساخته می‌شود؟

- کلید پیش‌فرض cache از **متد و URL** ساخته می‌شود؛ کوکی و کاربر در آن نیستند (RFC 9111 §2)
- دو راه حل: `private`، یا افزودن کوکی به کلید با `Vary` (اسلاید بعد)

درخواست	کلید cache	پاسخی که cache برمی‌گرداند
`GET /menu`	`GET /menu`	منوی روز، برای همه یکسان
`GET /balance` **با کوکی سارا**	`GET /balance`	موجودی سارا (ذخیره می‌شود)
`GET /balance` **با کوکی رضا**	`GET /balance`	باز هم موجودی سارا

> **cache دو درخواست را فقط از روی کلید از هم تشخیص می‌دهد.**

فیلد Vary چه چیزی را به کلید cache اضافه می‌کند؟

- پاسخ با Vary می‌گوید به کدام فیلد درخواست بستگی دارد (مثل زبان، فشرده‌سازی یا `Origin`)؛ cache **برای هر مقدار آن فیلد یک نسخه** نگه می‌دارد

header پاسخ	کلید cache	نتیجه
`Vary: Accept-Language`	`GET /` و زبان درخواست	یک نسخه برای هر زبان
`Vary: Cookie`	`GET /balance` و کوکی	یک نسخه برای هر کاربر؛ موجودی سارا فقط به سارا می‌رسد

- `Vary: Cookie` مشکل جدول قبل را حل می‌کند، ولی دادهٔ شخصی همه در cache مشترک می‌ماند و تقریباً دوباره به کار نمی‌رود؛ `private` بهتر است

> **هر فیلدی که به کلید اضافه شود، نسخه‌ها را بیشتر و سود cache را کمتر می‌کند.**

چه کسی تعیین می‌کند کدام نشانی cache شود؟

- **راه استاندارد:** سرور اصلی در هر پاسخ `Cache-Control` می‌فرستد و cacheهای مسیر از آن پیروی می‌کنند
- **راه دوم:** صاحب سایت در پنل CDN قاعده می‌نویسد، مثل «نشانی‌های زیر /static را یک روز نگه دار»؛ این قاعده می‌تواند بر header سرور مقدم شود
- بسیاری از CDNها پیش‌فرض از روی **پسوند نشانی** تصمیم می‌گیرند: Cloudflare فایل css و js و jpg را cache می‌کند و HTML را نه. پاسخ شخصی‌ای که `private` یا `no-store` ندارد و نشانی‌اش به css ختم شود، هم ذخیره می‌شود (Cloudflare Docs)

> **خطر وقتی است که header پاسخ و قاعدهٔ CDN با هم نخوانند.**

سناریوی واقعی: چگونه نشانی css نشست ChatGPT را لو داد؟

در فروردین ۱۴۰۲، Gal Nagli نشانی `/api/auth/session/x.css` را برای قربانی فرستاد. هر cache نشانی با پسوند css را ذخیره می‌کرد و سرور بخش x.css را نادیده می‌گرفت.

الف) سرور کوکی را بررسی نمی‌کرد

ب) cache از روی پسوند تصمیم گرفت

ج) کوکی قربانی دزدیده شد

د) مسیر بدون TLS بود

سناریوی واقعی: چگونه نشانی css نشست ChatGPT را لو داد؟

در فروردین ۱۴۰۲، Gal Nagli نشانی `/api/auth/session/x.css` را برای قربانی فرستاد. cache هر نشانی با پسوند css را ذخیره می‌کرد و سرور بخش x.css را نادیده می‌گرفت.

الف) سرور کوکی را بررسی نمی‌کرد	**ب)** cache از روی پسوند تصمیم گرفت	**ج)** کوکی قربانی دزدیده شد	**د)** مسیر بدون TLS بود

پاسخ: ب) پاسخ شخصی قربانی، با access token، در cache ماند؛ این حمله را web cache deception می‌نامند (SecurityWeek).

> **تصمیم به ذخیره کردن باید از headerهای پاسخ گرفته شود، نه از ظاهر نشانی.**

اگر یک header درخواست پاسخ را تغییر دهد ولی در کلید نباشد، چه می‌شود؟

- فرض کنید سرور نشانی فایل‌های JavaScript را از روی `X-Forwarded-Host` می‌سازد، ولی این فیلد در کلید cache نیست
- مهاجم یک بار با `X-Forwarded-Host: evil.example` درخواست می‌دهد و پاسخ آلوده **به همهٔ کاربران** می‌رسد؛ به این حمله web cache poisoning می‌گویند (PortSwigger)
- راه درست: یا این فیلد با `Vary` به کلید اضافه شود، یا سرور اصلاً از آن استفاده نکند

> **ورودی‌ای که پاسخ را تغییر می‌دهد، باید در کلید cache باشد.**

مرور cache: چهار پرسش را از حافظه پاسخ دهید

پرسش	سازوکار	header	اگر اشتباه شود
پاسخ تا کی تازه است؟	؟	؟	؟
تازه نیست؛ هنوز معتبر است؟	؟	؟	؟
مال همین درخواست و همین کاربر است؟	؟	؟	؟
کدام لایه آن را نگه دارد؟	؟	؟	؟

همهٔ این قرارداد در headerهاست: سرور در پاسخ شرط‌ها را اعلام می‌کند، و cache در درخواست بعدی از سرور می‌پرسد.

مرور cache: چهار پرسش را از حافظه پاسخ دهید

پرسش	سازوکار	header	اگر اشتباه شود
پاسخ تا کی تازه است؟	تاریخ مصرف	`Cache-Control: max-age` `Age`	کاربر نسخهٔ کهنه را می‌بیند
تازه نیست؛ هنوز معتبر است؟	پرسیدن با شناسهٔ نسخه	`ETag` `If-None-Match` `304`	کل پاسخ بی‌دلیل دوباره فرستاده می‌شود
مال همین درخواست و همین کاربر است؟	کلید cache	`Vary` `private`	پاسخ یک کاربر به کاربر دیگر می‌رسد
کدام لایه آن را نگه دارد؟	دستور جدا برای هر لایه	`private` `s-maxage` `no-store`	داده‌ای شخصی در cache مشترک می‌ماند

همهٔ این قرارداد در headerهاست: سرور در پاسخ شرط‌ها را اعلام می‌کند، و cache در درخواست بعدی از سرور می‌پرسد.

> **cache معامله‌ای میان سرعت و درستی است؛ هر header یکی از شرط‌های این معامله است.**

بخش ۴ از ۵

واسطه‌ها برای چه کسی کار می‌کنند؟

تا اینجا: cacheهای مشترکی که دیدیم، همه جایی بین کاربر و سرور اصلی قرار داشتند.

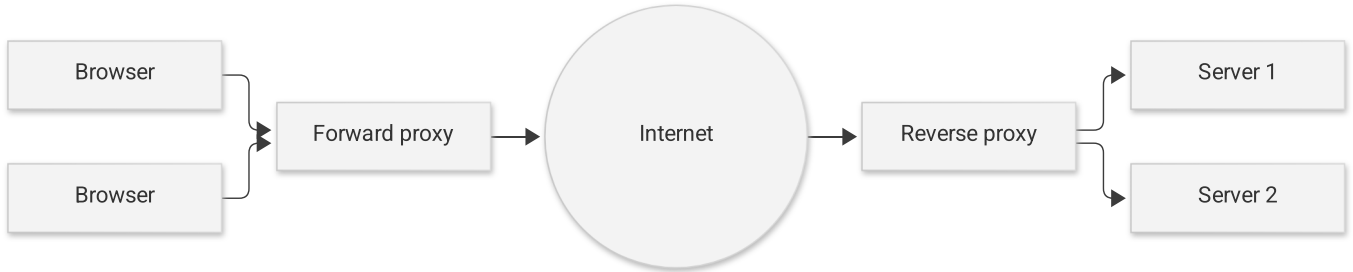
سؤال این بخش

هر واسطه برای چه کسی کار می‌کند، و وقتی ارتباط HTTPS است، چه چیزی را می‌بیند؟

05-intermediaries.todo

- [x] cache و تازگی
- [x] conditional request
- [x] کلید cache
- [>] **proxy و CDN** ◀
- [ ] جمع‌بندی

proxy برای چه کسی کار می‌کند؟

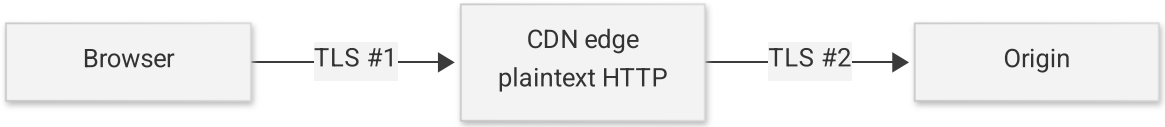


- forward proxy کنار کلاینت قرار دارد و کلاینت یا سازمانش آن را تنظیم کرده است؛ مثل proxy شبکهٔ یک شرکت
- reverse proxy کنار سرور قرار دارد و کلاینت **از وجود آن خبر ندارد**؛ کارش پخش بار بین سرورها، cache و باز کردن TLS است

> *A forward proxy acts on behalf of clients (or requesting hosts).*
>
> MDN, Proxy servers and tunneling

هر دو از یک نوع نرم‌افزارند؛ تفاوتشان در این است که چه کسی آن‌ها را آنجا گذاشته است.

چرا CDN یک reverse proxy توزیع‌شده است، و TLS کجا باز می‌شود؟



- CDN صدها reverse proxy در نقاط مختلف دنیا دارد و DNS یا anycast هر کاربر را به نزدیک‌ترین آن‌ها می‌فرستد (جلسهٔ ۱)
- سرور لبه برای ذخیره کردن پاسخ باید پیام HTTP را بخواند؛ پس **برای نام سایت گواهی دارد** و اتصال TLS را خودش باز می‌کند. از لبه تا سرور اصلی معمولاً یک اتصال TLS جداگانه برقرار می‌شود

> **قفل نوار نشانی فقط می‌گوید مسیر تا لبهٔ CDN رمز شده است، نه تا سرور اصلی.**

در اتصال HTTPS، چه کسانی کوکی نشست شما را می‌بینند؟

proxy دانشگاه برای سایت‌های HTTPS فقط یک تونل `CONNECT` باز می‌کند؛ proxy یک شرکت گواهی ریشهٔ خودش را روی رایانه‌ها نصب کرده است. سایت پشت CDN است.

الف) فقط سرور اصلی

ب) سرور اصلی و CDN

ج) سرور اصلی، CDN و proxy شرکت

د) همه، حتی ارائه‌دهندهٔ اینترنت

در اتصال HTTPS، چه کسانی کوکی نشست شما را می‌بینند؟

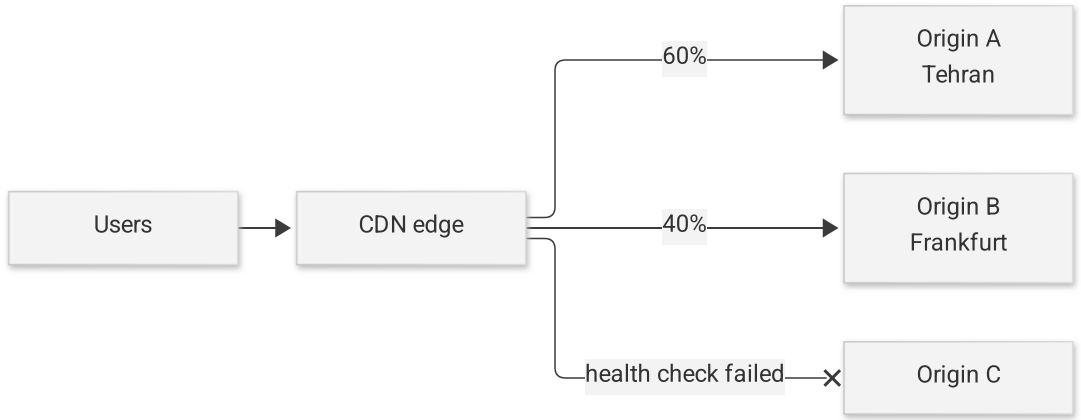
proxy دانشگاه برای سایت‌های HTTPS فقط یک تونل `CONNECT` باز می‌کند؛ proxy یک شرکت گواهی ریشهٔ خودش را روی رایانه‌ها نصب کرده است. سایت پشت CDN است.

الف) فقط سرور اصلی	**ب)** سرور اصلی و CDN	**ج)** سرور اصلی، CDN و proxy شرکت	**د)** همه، حتی ارائه‌دهندهٔ اینترنت

پاسخ: ج) proxy دانشگاه از درون تونل فقط نام میزبان را می‌بیند؛ CDN و proxy شرکت اتصال TLS را باز می‌کنند.

> **هر واسطه‌ای که اتصال TLS را باز کند، متن ساده را می‌بیند.**

CDN چگونه بار را بین چند سرور اصلی پخش می‌کند؟



- سایت بزرگ چند سرور اصلی در چند مرکز داده دارد. CDN تصمیم می‌گیرد هر درخواست به کدام سرور برود؛ به این کار load balancing می‌گویند
- CDN مدام سلامت سرورها را می‌سنجد (health check) و اگر سروری پاسخ ندهد، **ترافیک را به سرورهای سالم می‌فرستد** (failover)

CDN جلوی همهٔ سرورها نشسته است؛ پس خرابی یکی از آن‌ها را از چشم کاربر پنهان می‌کند.

CDN جز cache چه کارهای دیگری انجام می‌دهد؟

کار	چه می‌کند؟
محافظت در برابر DDoS	ترافیک حمله در صدها مرکز داده جذب می‌شود و به سرور اصلی نمی‌رسد
دیوارهٔ آتش وب (WAF)	درخواست‌های مخرب، مثل تزریق SQL، و botهای ناخواسته در لبه رد می‌شوند
مدیریت گواهی و پروتکل	گواهی سایت را صادر و تمدید می‌کند و HTTP/3 را در لبه فعال می‌کند
بهینه‌سازی محتوا	پاسخ را فشرده می‌کند و اندازه و قالب تصویر را تغییر می‌دهد
اجرای کد در لبه	برنامهٔ کوچک شما روی سرورهای لبه اجرا می‌شود؛ مثل Cloudflare Workers

CDN امروز فقط cache نیست؛ دروازه‌ای است که همهٔ ترافیک سایت از آن می‌گذرد.

شبکه‌های CDN شناخته‌شده کدام‌اند؟

CDN	ویژگی
Cloudflare	پرکاربردترین CDN، با حدود ۲۸٪ همهٔ وب‌سایت‌ها (W3Techs، آبان ۱۴۰۵)؛ نسخهٔ رایگان دارد
Akamai	نخستین CDN تجاری بزرگ (۱۹۹۸)؛ بیشتر برای شرکت‌های بزرگ و ویدیو
CloudFront	CDN خدمات ابری آمازون (AWS)
Fastly	امکان نوشتن منطق دلخواه در لبه، و پاک کردن cache در کسری از ثانیه
آروان‌کلاد	CDN ایرانی با سرورهای لبه در چند شهر ایران و خارج؛ DNS و محافظت در برابر DDoS هم دارد

> **قطعی Cloudflare در آبان ۱۴۰۴ هزاران سایت را با هم از کار انداخت.**

محدودیت نرخ (rate limit) را بر اساس کدام نشانی IP بگذاریم؟

nginx شما (`10.0.0.5`) نشانی طرفی را که مستقیماً به آن وصل شده، به انتهای فیلد می‌افزاید؛ `198.51.100.20` لبهٔ CDN است.

```
X-Forwarded-For: 6.6.6.6, 203.0.113.7, 198.51.100.20
```

الف) `6.6.6.6`

ب) `203.0.113.7`

ج) `198.51.100.20`

د) `10.0.0.5`

محدودیت نرخ (rate limit) را بر اساس کدام نشانی IP بگذاریم؟

nginx شما (`10.0.0.5`) نشانی طرفی را که مستقیماً به آن وصل شده، به انتهای فیلد می‌افزاید؛ `198.51.100.20` لبهٔ CDN است.

```
X-Forwarded-For: 6.6.6.6, 203.0.113.7, 198.51.100.20
```

الف) `6.6.6.6`

ب) `203.0.113.7`

ج) `198.51.100.20`

د) `10.0.0.5`

پاسخ: ب) آخرین ورودی را nginx و ورودی قبلی را CDN نوشته؛ ورودی سمت چپ را خود کلاینت گذاشته است.

> **فقط به ورودی‌هایی اعتماد کنید که واسطه‌های خودتان نوشته‌اند.**

Cloudflare Workers: برنامهٔ شما روی سرورهای لبه

- تابعی به زبان JavaScript که در یک V8 isolate روی سرورهای لبهٔ Cloudflare و **پیش از رسیدن درخواست به سرور اصلی** اجرا می‌شود (از اسفند ۱۳۹۶)
- کاربردها: تغییر مسیر، بررسی token، ساختن پاسخ بدون سرور اصلی، و با Workers AI اجرای مدل

```
export default {
  fetch: (req) => new URL(req.url).pathname === '/old-menu'
    ? Response.redirect('https://buffet.example/menu', 301)
    : fetch(req),   // else: to the origin
}
```

> **با اجرای کد در لبه، CDN خودش هم پاسخ می‌سازد.**

proxy شبکه از ترافیک HTTPS چه چیزی را می‌بیند؟

- مرورگر به proxy می‌گوید `CONNECT www.example.com:443` و از آن به بعد proxy فقط بایت‌های رمزشده را جابه‌جا می‌کند. proxy نام میزبان و درگاه را می‌بیند، ولی **نه مسیر را و نه کوکی را** (RFC 9110 §9.3.6)
- در TLS interception، سازمان گواهی ریشهٔ خودش را روی رایانه‌ها نصب می‌کند و proxy دو اتصال TLS جداگانه برقرار می‌کند؛ درست مثل CDN، ولی به نمایندگی از سازمان
- فیلد استاندارد `Forwarded` (RFC 7239) جانشین `X-Forwarded-For` است و همان مشکل اعتماد را دارد. فیلد `Via` نشان می‌دهد پیام از کدام واسطه‌ها گذشته است

> **تونل CONNECT محتوا را باز نمی‌کند؛ ولی با گواهی ریشهٔ نصب‌شده، proxy همه‌چیز را می‌بیند.**

سناریوی واقعی: قطعی Cloudflare در ۲۷ آبان ۱۴۰۴ چگونه رخ داد؟

- در ۲۷ آبان ۱۴۰۴ (۱۸ نوامبر ۲۰۲۵)، تغییری در مجوزهای یک پایگاه‌داده باعث شد فایل ویژگی‌های Bot Management در Cloudflare ردیف‌های تکراری بگیرد و حجمش دو برابر شود
- این فایل از سقفی که proxy اصلی Cloudflare برایش در نظر گرفته بود بزرگ‌تر شد. از حدود ساعت ۱۱:۲۰ به وقت UTC و نزدیک به سه ساعت، بسیاری از سایت‌ها **خطای 5xx** برگرداندند (پست‌مورتم Cloudflare)

> **هر چه واسطهٔ مشترک بزرگ‌تر باشد، خطای آن هم‌زمان سایت‌های بیشتری را از کار می‌اندازد.**

سناریوی واقعی: ماجرای Cloudbleed چه چیزی را دربارهٔ CDN نشان داد؟

- در ۳۰ بهمن ۱۳۹۵ (۱۸ فوریهٔ ۲۰۱۷)، Tavis Ormandy از Google Project Zero متوجه شد پاسخ‌های بعضی سایت‌های پشت Cloudflare تکه‌هایی از حافظهٔ سرور لبه را در خود دارند
- خطایی در parser HTML سرورهای لبه، از مهر ۱۳۹۵، داده‌هایی از حافظه را وارد پاسخ‌ها می‌کرد؛ از جمله **کوکی، token و گذرواژهٔ کاربران سایت‌های دیگر**. بخشی از این داده در cache موتورهای جست‌وجو هم ماند و بعداً پاک شد (گزارش Cloudflare)

> **این داده‌ها فقط به این دلیل لو رفتند که لبهٔ CDN متن ساده را در حافظه داشت.**

بخش ۵ از ۵

جمع‌بندی

تا اینجا: در این جلسه دیدیم هر واسطه پاسخ را تا چه زمانی، برای چه کسی و به نمایندگی از چه کسی نگه می‌دارد.

سؤال این بخش

چه کسی بین شما و سرور نشسته است، و چه چیزی را می‌بیند؟

05-intermediaries.todo

[x] cache و تازگی
[x] conditional request
[x] کلید cache
[x] proxy و CDN
[>] جمع‌بندی ▶

نقشهٔ مسیر را از حافظه کامل کنید

از مرورگر تا سرور	پاسخ را نگه می‌دارد؟	برای همه یا یک نفر؟	به نمایندگی از	با HTTPS متن ساده را می‌بیند؟
مرورگر cache	؟	؟	؟	؟
شبکهٔ شما proxy	؟	؟	؟	؟
ارائه‌دهندهٔ اینترنت	؟	؟	؟	؟
لبهٔ CDN	؟	؟	؟	؟
سرور reverse proxy	؟	؟	؟	؟
سرور اصلی	؟	؟	؟	؟

اول ستون‌ها را از حافظه پر کنید، سپس پاسخ خود را با جدول مقایسه کنید.

نقشهٔ مسیر را از حافظه کامل کنید

از مرورگر تا سرور	پاسخ را نگه می‌دارد؟	برای همه یا یک نفر؟	به نمایندگی از	با HTTPS متن ساده را می‌بیند؟
cache مرورگر	بله	یک نفر (خصوصی)	کاربر	بله، درون مرورگر است
proxy شبکهٔ شما	گاهی	همه (مشترک)	کاربر یا سازمان	فقط با گواهی ریشهٔ نصب‌شده
ارائه‌دهندهٔ اینترنت	نه	—	هیچ‌کدام؛ فقط بسته را می‌برد	نه؛ معمولاً فقط IP و نام سایت
لبهٔ CDN	بله	همه (مشترک)	صاحب سایت	بله
reverse proxy سرور	گاهی	همه (مشترک)	صاحب سایت	بله
سرور اصلی	منبع پاسخ	—	خودش	بله

اول ستون‌ها را از حافظه پر کنید، سپس پاسخ خود را با جدول مقایسه کنید.

این جدول پاسخ پرسش اصلی جلسه است.

این مفاهیم کدام تصمیم‌های مهندسی را بهتر می‌کنند؟

مدل ذهنی	تصمیم مهندسی
پاسخ تازه بدون پرسیدن از سرور تحویل داده می‌شود	عمر پاسخ را با `max-age` اعلام کنید؛ «ذخیره نکن» یعنی `no-store`
پاسخ `304` بدنه ندارد، ولی یک RTT می‌گیرد	فایل ثابت را با نشانی دارای هش و `immutable` منتشر کنید
کلید cache کاربر را نمی‌شناسد	پاسخ شخصی را `private` کنید و قاعدهٔ cache را بر اساس پسوند ننویسید
واسطه‌ای که TLS را باز کند، متن ساده را می‌بیند	فقط به ورودی‌هایی از `X-Forwarded-For` اعتماد کنید که واسطه‌های خودتان نوشته‌اند

جلسهٔ بعد: اگر سرور بخواهد خودش خبری بدهد، چه باید کرد؟

تمرین ۳: رزومهٔ شما روی لبهٔ اینترنت

چالش یک صفحهٔ رزومهٔ ساده روی Cloudflare Workers، با نشانی‌ای در `workers.dev` که شمارهٔ دانشجویی شما را دارد. صفحه باید به متدهای `GET` و `HEAD` و `POST` و `OPTIONS` درست پاسخ دهد و برای هر مسیر `Cache-Control` و `ETag` مناسب بفرستد. آزمون‌ها روی سایت زندهٔ شما اجرا می‌شوند.

سؤال اگر صفحه را با `max-age=300` بفرستید و یک دقیقه بعد رزومه را عوض کنید، کسی که صفحه را دیده بود چه می‌بیند؟ و فایل css با نام دارای هش چطور؟

پل اگر بخواهید صفحهٔ باز رزومه همان لحظه از پیام تازه خبردار شود، چه باید کرد؟ این موضوع جلسهٔ ۶ است.

مهلت: یک هفته

ie1405-scu.github.io/exercises/03-edge-cv/

نقشهٔ فصل ۱: کجای راهیم؟

ch1/syllabus.todo

- [x] جلسهٔ ۱: از URL تا سرور — URL و Origin و DNS، و TLS از دور
- [x] جلسهٔ ۲: یک تراکنش HTTP — متد و کد وضعیت و header
- [x] جلسهٔ ۳: state و هویت — cookie و CORS و CSRF
- [x] جلسهٔ ۴: زیربنای HTTP — TCP و QUIC و TLS
- [>] **جلسهٔ ۵: واسطه‌ها — cache و proxy و CDN** ▶
- [ ] جلسهٔ ۶: فراتر از request/response — SSE و WebSocket
- [ ] جلسهٔ ۷: کلاینت‌های خودکار — agent و MCP

هر جلسه یک حلقه از همان زنجیرهٔ ابتدای فصل را بررسی می‌کند.

برای کنجکاوی بیشتر (۱ از ۲)

۱ **اگر هزار کاربر هم‌زمان صفحه‌ای را بخواهند که در cache مشترک کهنه شده است، چند درخواست به سرور اصلی می‌رسد؟**

سرنخ: دربارهٔ request collapsing در راهنمای HTTP caching در MDN بخوانید.

۲ **فایل اشتباهی را با `max-age` یک‌ساله منتشر کرده‌اید. چگونه آن را از cache مرورگر کاربران پاک کنید؟**

سرنخ: header `Clear-Site-Data` را ببینید، و بررسی کنید چرا نشانی تازه راه مطمئن‌تری است.

۳ **چرا پاسخ `POST` معمولاً cache نمی‌شود، با اینکه RFC آن را ممنوع نکرده است؟**

سرنخ: بخش POST در RFC 9110 §9.3.3 و شرط `Content-Location` را ببینید.

برای کنجکاوی بیشتر (۲ از ۲)

۴ آیا می‌شود کلید خصوصی گواهی سایت را به CDN نداد، ولی باز هم TLS را در لبه باز کرد؟

سرنخ: دربارهٔ Keyless SSL در Cloudflare و Delegated Credentials (RFC 9345) بخوانید.

۵ اگر سرور اصلی بخواهد پاسخ فقط در CDN ذخیره شود و در مرورگر نه، از کدام header استفاده می‌کند؟

سرنخ: `CDN-Cache-Control` را در RFC 9213 ببینید.

۶ APIهای مدل‌های زبانی هم «cache» دارند (prompt caching). این cache چه شباهت و چه تفاوتی با cache در HTTP دارد؟

سرنخ: کلید و عمر و دامنهٔ اشتراک هر دو را مقایسه کنید: چه کسی کلید را می‌سازد و چه کسانی از یک نسخه استفاده می‌کنند؟

منابع

- منبع درسی: کتاب *Computer Networking* نوشتهٔ Kurose و Ross، بخش‌های ۲.۲.۵ و ۲.۶
- RFC 9110 §13: Conditional Requests · RFC 9111: HTTP Caching
- RFC 5861 · RFC 8246: immutable · RFC 7239: Forwarded
- MDN: Proxy servers and tunneling · MDN: HTTP caching
- Age · Vary · MDN: X-Forwarded-For
- Workers · Cloudflare: Load Balancing
- W3Techs: Cloudflare vs Fastly
- Valve: Steam Christmas incident
- SecurityWeek: ChatGPT web cache deception
- PortSwigger: Web cache poisoning
- Cloudbleed · Cloudflare: 18 November 2025 outage
- Wikimedia: How crawlers impact operations